FedSelfSelect

Identification-free rare-class learning in federated classification

1 Problem

Federated classification keeps records on K clients and fits one shared model w. Label-aware methods such as Stratify do not average every local epoch. They schedule a turn per class and train on samples of that class. The server must then know who holds class c, because it names the client that trains.

That naming is the failure we study. One class is rare: only $k_{\text{rare}} \ll K$ clients hold it. If the server publishes noisy label metadata and then routes from that metadata, the rare class is the first to break.

Randomized-response presence bits, as in pp-CFL, flip many zeros to ones. With two true holders among a hundred clients, false positives drown the rare bit. The server sends the rare turn to a non-holder. The turn is lost. Rare accuracy collapses. Overall accuracy stays high, because common classes still have many holders.

Laplace counts, as in Stratify, often still rank the true holders. A local count of thirty survives noise of scale $1/\varepsilon$. Rare accuracy then stays high. The cost is identification: the server selected those clients, so it knows who holds the rare class.

The problem is therefore not "raise rare accuracy above Stratify." The problem is to train the rare class *without the server choosing holders*. FedAvg avoids naming people, but it never inspects labels, so the rare class is almost unused. FedSelfSelect keeps class turns, drops the naming step, and hides the uploader set.

2 Method

Holders of class c always upload a class-c gradient. Non-holders upload a zero with probability q. The server sees only a secure sum, then divides by a noisy holder count, not by the number of uploaders. Dummy zeros do not change the sum. After all C classes, the model takes one step on the class-mean gradient.

Algorithm 1: FedSelfSelect

Input: clients $\{1, \dots, K\}$; classes $\{1, \dots, C\}$; steps T; learning rate η; batch size B; privacy budget ε; dummy rate q

Output: global model w

1 $w \leftarrow$ random initialization

`// once: server learns how many holders, not who`

2 **for** $c = 1$ **to** C **do**

3 $\quad n_c \leftarrow \texttt{SecAgg}\big(\{\mathbf{1}[c \in \mathcal{D}_k]\}_{k=1}^{K}\big)$

4 $\quad \tilde{n}_c \leftarrow \max\big(n_c + \texttt{Lap}(1/\varepsilon),\, 1\big)$

5 **for** $t = 1$ **to** T **do**

6 $\quad G \leftarrow 0$

7 $\quad$ **for** $c = 1$ **to** C in random order **do**

8 $\quad\quad$ broadcast $(w,\, c)$ to all clients

9 $\quad\quad \mathcal{U}_c \leftarrow \emptyset$

10 $\quad\quad$ **for** $k = 1$ **to** K **in parallel do**

11 $\quad\quad\quad$ **if** $c \in \mathcal{D}_k$ **then**

12 $\quad\quad\quad\quad$ sample minibatch $\mathcal{B}_k$ from class-c samples in $\mathcal{D}_k$

13 $\quad\quad\quad\quad g_k \leftarrow \nabla \ell(w; \mathcal{B}_k)$

14 $\quad\quad\quad\quad \mathcal{U}_c \leftarrow \mathcal{U}_c \cup \{k\}$ `// holders always upload`

15 $\quad\quad\quad$ **else**

16 $\quad\quad\quad\quad$ **if** $\texttt{Bernoulli}(q) = 1$ **then**

17 $\quad\quad\quad\quad\quad g_k \leftarrow 0$

18 $\quad\quad\quad\quad\quad \mathcal{U}_c \leftarrow \mathcal{U}_c \cup \{k\}$ `// dummy zero`

19 $\quad\quad S_c \leftarrow \texttt{SecAgg}(\{g_k : k \in \mathcal{U}_c\})$

20 $\quad\quad G \leftarrow G + S_c/\tilde{n}_c$ `// not` $|\mathcal{U}_c|$

21 $\quad w \leftarrow w - \eta \cdot G/C$

22 **return** w

Lines	Who	Role
2–4	Server	Receives only the holder *count* per class. Noise sits on $\tilde{n}_c$.
8	Server	Broadcasts (w, c). Does not name a client.
11–14	Holder	Computes a minibatch gradient and always uploads.
15–18	Non-holder	Uploads 0 with probability q; otherwise silent.
19–20	Server	Receives the secure sum and divides by $\tilde{n}_c$, not $\lvert\mathcal{U}_c\rvert$.
21	Server	Averages the C class means and takes one step.

3 Key equations

Let $\mathcal{D}_k$ be client k's labeled set and

$$\mathcal{H}_c = \{k : c \in \mathcal{D}_k\}, \qquad n_c = |\mathcal{H}_c| = \sum_{k=1}^{K} \mathbf{1}[c \in \mathcal{D}_k]. \tag{1}$$

Equation (1) is the object the server is allowed to see, as a sum, not as a name list. Secure aggregation returns n_c once. Laplace noise is applied only to that scalar:

$$\tilde{n}_c = \max\big(n_c + \text{Lap}(1/\varepsilon),\, 1\big). \tag{2}$$

The noisy count is a divisor. It is not a routing score. The no-noise ablation uses $\tilde{n}_c \leftarrow n_c$.

On a class-c turn the server broadcasts (w, c). Holder $k \in \mathcal{H}_c$ draws a minibatch $\mathcal{B}_k$ from its class-c samples and sends

$$g_k = \nabla_w \ell(w; \mathcal{B}_k). \tag{3}$$

A non-holder joins the uploader set with probability q and sends a zero:

$$k \in \mathcal{U}_c \text{ if } k \in \mathcal{H}_c \text{ or } \big(k \notin \mathcal{H}_c \text{ and Bernoulli}(q) = 1\big). \tag{4}$$

The server receives only the secure sum

$$S_c = \sum_{k \in \mathcal{U}_c} g_k. \tag{5}$$

Dummy coordinates are zero, so they do not change S_c. The class vector used in the step is the holder mean, not the uploader mean:

$$\bar{g}_c = \frac{S_c}{\tilde{n}_c} \qquad (\text{not } S_c/|\mathcal{U}_c|). \tag{6}$$

Dividing by $|\mathcal{U}_c|$ would shrink the rare gradient whenever dummy traffic is large. Dividing by $\tilde{n}_c$ keeps the scale of a mean over holders. After a random permutation of the C classes,

$$w \leftarrow w - \eta \cdot \frac{1}{C} \sum_{c=1}^{C} \bar{g}_c. \tag{7}$$

Equation (7) is an unweighted mean over classes, not a sample-size weighted mean over clients. Each class, including the rare class, contributes $1/C$ of the step. Expected uploads on a class-c turn are

$$\mathbb{E}[|\mathcal{U}_c|] = n_c + q(K - n_c). \tag{8}$$

With $n_c = 2$, $K = 100$, and $q = 0.1$, a rare turn has about twelve uploaders.

4 Loss

The predictor f_w is a one-hidden-layer ReLU MLP with width 64 and a softmax head. The local loss is multiclass cross-entropy. For a labeled pair (x, y),

$$\ell(w; x, y) = -\log\big[\text{softmax}(f_w(x))\big]_y. \tag{9}$$

A minibatch of size B (or all class-c samples, if fewer than B) uses the mean of (9). Rare holders have thirty samples, so a rare minibatch is those thirty points. The reported metrics are not RMSE. The task is classification: overall accuracy on a natural-mix test set, rare-class recall, and balanced accuracy (mean per-class recall on a class-balanced test set).

FedAvg, used only as a label-blind baseline, minimizes the same loss with local SGD on mixed minibatches and sample-size weighted aggregation. It does not use class turns.

5 What is guaranteed

The claims below are identities of Algorithm 1. They are not a rare-accuracy rate and not differential privacy of the trained weights w_T.

Proposition 1 (Dummy zeros do not change the sum)**.** *If every holder uploads* (3) *and every dummy coordinate is* 0*, then* $S_c = \sum_{k \in \mathcal{H}_c} g_k$.

The uploader set $\mathcal{U}_c$ is therefore not equal to $\mathcal{H}_c$, but the sum is still the holder sum. Under ideal secure aggregation the server does not see which summands were zero.

Proposition 2 (Noisy divisor, not noisy routing)**.** *If $\tilde{n}_c = n_c$, then $\bar{g}_c$ is the uniform mean of the holder gradients. Laplace noise in* (2) *perturbs only that divisor. It does not choose who trains.*

A count has sensitivity one, so $n_c + \mathrm{Lap}(1/\varepsilon)$ is ε-differentially private for that count. Flooring at 1 is post-processing. This statement is only about $\{\tilde{n}_c\}_{c=1}^{C}$. It is not a claim that w_T is differentially private, and it is not a bound on rare recall.

We do not prove convergence of (7). We do not prove a K/k_{rare} rate. Communication in (8) is larger than one-holder routing. That cost is a limitation, not the objective.

6 Experimental setup

The partition is an experimental setup, not an assumption about deployed networks. We use three tabular classification sets: Covertype (C=7, d=54), Statlog Shuttle (C=4, d=9), and Dry Bean (C=7, d=16). Classes with fewer than 100 samples are dropped. The rare class is the smallest class that remains.

A *holder* of class c is a client that has at least one sample of class c. There are K=100 clients. Each client holds two or three common classes, split by a Dirichlet(1) draw. The rare class has exactly two holders. When we split the data we give each of those two holders 30 rare samples. That 30 is the local rare *dataset* size. It is not the minibatch size. Common classes use minibatches of B=64. A rare holder has only 30 points, so the rare gradient uses all 30. Common classes are capped at 6,000 training samples. No point appears in more than one of train, balanced test, and natural-mix test. The balanced test uses up to 400 points per class and is used for rare and balanced accuracy. The natural-mix test keeps the original class frequencies and is used for overall accuracy.

There are two training clocks. They are not the same quantity, and 1,500 is not 150×10. Class-turn methods (FedSelfSelect, Stratify, pp-CFL, Oracle) run T=1,500 steps. One step visits every class once, in random order. FedAvg does not use class turns. It runs 150 rounds, picks 10 of the 100 clients each round, and each picked client takes 10 local SGD steps on mixed minibatches. The product $150 \times 10 \times 10$ belongs to FedAvg only.

The model is the MLP above, learning rate η=0.1. Private methods use ε=0.5 in the main tables. FedSelfSelect uses dummy rate q=0.1. Means are over seeds $\{0, 1, 2\}$.

Routing baselines share the class-turn loop. Oracle samples a true holder. Stratify here is Laplace counts used as a routing score, not CKKS. pp-CFL here is randomized-response presence bits used as a routing score, not the full clustering pipeline. A routed client with no class-c samples returns no gradient, and that turn is lost. Oracle and Stratify therefore upload C vectors per step. FedSelfSelect uploads (8) per class.

Table 1: Data split. Shared by every method.

Item	Value
Clients K	100
Rare holders (clients who have the rare class)	2
Rare samples on each of those 2 clients	30
Common classes per client	2–3
Minibatch B (common classes)	64
Hidden width / learning rate η	64 / 0.1
Dummy rate q / privacy ε (main tables)	0.1 / 0.5
Seeds	$0, 1, 2$

Covertype is the setting where overall scores match and rare scores split. Shuttle is a second dataset with the same split. Dry Bean is a case where the rare class is easy, so routing errors barely move rare accuracy.

Table 2: Two different training loops. Do not multiply one into the other.

Loop	Methods	Budget
Class turn	FedSelfSelect, Stratify, pp-CFL, Oracle	1,500 steps
FedAvg family	FedAvg, FedProx, SCAFFOLD, FedDyn, MOON	150 rounds $\times$ 10 clients $\times$ 10 local steps

7 Accuracy

Means over three seeds, in percent. Noisy methods use $\varepsilon = 0.5$.

Table 3: Accuracy (%). Covertype, Shuttle, and Dry Bean. Private methods use $\varepsilon = 0.5$.

	Covertype			Shuttle			Dry Bean		
Method	overall	rare	bal.	overall	rare	bal.	overall	rare	bal.
FedAvg	57.9	0.0	55.8	98.3	2.8	75.3	90.7	54.3	86.8
FedProx	57.9	0.0	55.8	98.3	2.1	75.1	90.6	52.7	86.6
SCAFFOLD	62.1	0.0	61.0	98.8	9.0	77.0	92.7	100.0	93.5
FedDyn	58.2	0.0	55.9	98.2	0.7	74.7	90.6	49.1	86.1
MOON	57.9	0.0	56.0	98.4	2.1	75.1	90.6	52.7	86.7
pp-CFL	62.3	1.2	59.2	98.2	67.4	91.4	92.0	99.3	93.0
Stratify	62.2	70.7	70.8	98.6	92.4	97.7	92.8	99.5	93.4
FedSelfSelect	61.9	77.3	71.6	98.6	95.1	98.4	92.3	99.5	93.2
FedSelfSelect (no noise)	61.5	80.0	71.5	98.6	95.1	98.4	92.3	99.8	93.3
Oracle	61.6	79.6	71.6	98.6	93.8	98.1	92.6	99.8	93.5

Table 4: Rare-class accuracy only ($\varepsilon = 0.5$).

Method	Covertype	Shuttle	Dry Bean
FedAvg	0.0	2.8	54.3
FedProx	0.0	2.1	52.7
SCAFFOLD	0.0	9.0	100.0
FedDyn	0.0	0.7	49.1
MOON	0.0	2.1	52.7
pp-CFL	1.2	67.4	99.3
Stratify	70.7	92.4	99.5
FedSelfSelect	77.3	95.1	99.5
Oracle	79.6	93.8	99.8

Label-blind aggregators (FedAvg, FedProx, SCAFFOLD, FedDyn, MOON) keep overall accuracy and miss the rare class on Covertype. SCAFFOLD raises overall a little; rare recall stays 0.0. The rare column is the object of the problem. On Covertype, bit routing loses the rare class. Count routing keeps it and names holders. FedSelfSelect keeps it without naming holders. The gap versus Stratify is not the reason to use the method. Dry Bean does not separate label-aware methods; SCAFFOLD can fit BOMBAY without class turns because that class is easy.

8 Communication

Counts are model-sized uploads in one run. FedAvg: 150 rounds. All other methods: 1,500 steps. K=100, q=0.1.

*FedAvg is 150 $\times$ 10. Other methods scale by 1,500 steps. Shuttle has four classes; the others have seven. With q=0.1, FedSelfSelect uses 0.42$\times$ the all-upload cost on Covertype and 0.65$\times$ on Shuttle. Relative to

Table 5: Total uploads.

Method	Covertype	Shuttle	Dry Bean
FedAvg / FedProx / SCAFFOLD / FedDyn / MOON	1,500*	1,500*	1,500*
pp-CFL	10,500	6,000	10,500
Stratify	10,500	6,000	10,500
Oracle	10,500	6,000	10,500
FedSelfSelect (q=1)	1,050,000	600,000	1,050,000
FedSelfSelect (q=0.1)	444,750	392,100	438,000

Table 6: Uploads per step (totals above divided by 1,500; FedAvg is 10 per round).

Method	Covertype	Shuttle	Dry Bean
FedAvg family	10*	10*	10*
pp-CFL / Stratify / Oracle	7	4	7
FedSelfSelect (q=1)	700	400	700
FedSelfSelect (q=0.1)	297	261	292

Stratify that is still $42\times$ and $65\times$. Parameter counts are 3,975 (Covertype), 900 (Shuttle), and 1,543 (Dry Bean). Bytes scale as (uploads)$\times$(parameters).

9 Name

The working title FedSelfSelect is too close to FedSelect. Three acronyms of the method are recommended instead: FedHBT (Holder-Blind Turn), FedNCT (Nameless Class Turn), and FedBCT (Broadcast Class Turn).